Bluesky and the AT Protocol: Usable Decentralized Social Media

Martin Kleppmann
martin.kleppmann@cst.cam.ac.uk
University of Cambridge
Cambridge, UK

Paul Frazee
paul@blueskyweb.xyz
Bluesky Social PBC
Seattle, WA, USA

Jake Gold
jake@blueskyweb.xyz
Bluesky Social PBC
Mountain View, CA, USA

Jay Graber
jay@blueskyweb.xyz
Bluesky Social PBC
Seattle, WA, USA

Daniel Holmgren
daniel@blueskyweb.xyz
Bluesky Social PBC
Springfield, MO, USA

Devin Ivy
devin@blueskyweb.xyz
Bluesky Social PBC
Cambridge, MA, USA

Jeromy Johnson
why@blueskyweb.xyz
Bluesky Social PBC
Seattle, WA, USA

Bryan Newbold
bryan@blueskyweb.xyz
Bluesky Social PBC
Seattle, WA, USA

Jaz Volpert
jaz@blueskyweb.xyz
Bluesky Social PBC
San Francisco, CA, USA

Abstract

Bluesky is a new social network built upon the AT Protocol, a decentralized foundation for public social media. It was launched in private beta in February 2023, and has grown to over 10 million registered users by October 2024. In this paper we introduce the architecture of Bluesky and the AT Protocol, and explain how the technical design of Bluesky is informed by our goals: to enable decentralization by having multiple interoperable providers for every part of the system; to make it easy for users to switch providers; to give users agency over the content they see; and to provide a simple user experience that does not burden users with complexity arising from the system's decentralized nature. The system's openness allows anybody to contribute to content moderation and community management, and we invite the research community to use Bluesky as a dataset and testing ground for new approaches in social media moderation.

CCS Concepts

• **Information systems** → **Social networking sites**; • **Social and professional topics** → **Centralization / decentralization**; • **Networks** → *Social media networks.*

Keywords

social networks; decentralization; federation

ACM Reference Format:
Martin Kleppmann, Paul Frazee, Jake Gold, Jay Graber, Daniel Holmgren, Devin Ivy, Jeromy Johnson, Bryan Newbold, and Jaz Volpert. 2024. Bluesky and the AT Protocol: Usable Decentralized Social Media. In *Proceedings of the ACM Conext-2024 Workshop on the Decentralization of the Internet (DIN '24), December 9–12, 2024, Los Angeles, CA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3694809.3700740

This work is licensed under a Creative Commons Attribution 4.0 International License.
DIN '24, December 9–12, 2024, Los Angeles, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1252-4/24/12
https://doi.org/10.1145/3694809.3700740

1 Introduction

Over the last two decades, social media has evolved into a "digital town square" and a cornerstone of civic life [3]. When a social media service is under the full control of a single corporation, it may change its policies on the whim of its leaders [35]. Its operations are opaque (e.g. regarding which content is recommended to users), and its users lack agency over their user experience. As a result, there has been increasing interest in decentralized social networks, of which the *fediverse* around the ActivityPub protocol [22] and the Mastodon software [25] is perhaps the best known.

However, decentralization also introduces new challenges. For example, in the case of Mastodon, a user needs to choose a server when creating an account. This choice is significant because the server name becomes part of the username; migrating to another server implies changing username, and preserving one's followers during such a migration requires the cooperation of the old server. If a server is shut down without warning, accounts on that server cannot be recovered – a particular risk with volunteer-run servers. In principle, a user can host their own server, but only a small fraction of social media users have both the technical skills and the inclination to do so.

The distinction between servers in Mastodon introduces complexity for users that does not exist in centralized services. For example, a user viewing a thread of replies in the web interface of one server may see a different set of replies compared to viewing the same thread on another server, because a server only shows those replies that it knows about [1]. As another example, when viewing the web profile of an account on another server, clicking the "follow" button does not simply follow that account; instead, the user needs to enter the hostname of their own server and be redirected to a URL on their home server before they can follow the account. In our opinion, it is undesirable to burden users with such complexity arising from the federated architecture.

In this paper we introduce the *AT Protocol* (atproto), a decentralized foundation for social networking, and *Bluesky*, a Twitter-style microblogging app built upon it. A core design goal of atproto and Bluesky is to enable a user experience of the same or better quality as centralized services, while being open and decentralized on a

technical level. The AT Protocol is designed such that for every part of the system there can be multiple competing operators providing interoperable services, making it easy to switch providers.

Decentralization alone is not able to solve some of the thorniest problems of social media, such as misinformation, harassment, and hate speech [31]. However, by opening up the internals of a service to the public, decentralization can enable a marketplace of approaches to these problems [24]. For example, Bluesky allows anybody to run moderation services that make subjective decisions of selecting desirable content or flagging undesirable content, and users can choose which moderation services they want to subscribe to. Moderation services are decoupled from hosting providers, making it easy for users to switch moderation services until they find ones that match their preferences. We hope that this architectural openness enables communities to develop their own approaches to managing problematic content, independently of what hosting providers implement [24].

For example, researchers wanting to identify disinformation campaigns can easily get access to all content being posted, the social graph, and user profiles on Bluesky. If they are able construct an algorithm to label suspected disinformation, they can publish their labels in real time, and users who wish to see those labels can enable them in their client. One goal of this paper is to bring Bluesky and the AT Protocol to the attention of researchers working on such algorithms, and to invite them to use the rapidly growing dataset of Bluesky content as a basis for their work.

2 The Bluesky Social App

Bluesky is a microblogging application in the style of Twitter/X. The "official" client app is available on iOS, Android, and the web; several independently developed client apps are also available, such as Graysky [29] and deck.blue [16]. Users can make public posts containing up to 300 characters of text, up to four images, and short-form video. They can interact with posts by replying, reposting, or liking, and they can follow other users. By default users have two feeds: one showing posts by followed accounts in reverse chronological order, and one recommending popular content across the network. Users can also choose from many alternative feeds that show content on various topics, as explained below.

Bluesky launched an invite-only beta in February 2023, and has grown to over 10 million registered users in the following 20 months. Bluesky Social PBC (a public-benefit corporation) develops the official client app and operates the core services; the client and several server-side components are open source, dual-licensed under the MIT and Apache 2.0 licenses [13]. The protocols they use are defined by open specifications [11]. Several parts of the system, such as feed generators and alternative clients [10] are developed and operated by independent third parties.

User Handles. Like on Twitter/X, a Bluesky user has two names: the *display name* can be almost any string, and the *handle* needs to uniquely identify a user. A handle, prefixed with an @ sign, is used to mention another user in a post.

The need for handles to be unique creates challenges in decentralized systems, since it requires an authority that determines which handle is assigned to which user. Mastodon's approach is to include the server name in the handle, which makes it difficult to move to another server. An alternative would be to use a blockchain-based naming system, such as the Ethereum Name System (ENS) [15]; this has the disadvantage of requiring the user to buy cryptocurrency in order to create an account, which we wanted to avoid.

Instead, Bluesky and atproto use DNS domain names as handles. Users can sign up for a subdomain of `.bsky.social` for free. If a user already owns a domain name, they can claim it as their Bluesky handle by adding a DNS record or by hosting a file under a `/.well-known/` HTTPS URL on that domain [23]. Users can also buy a new domain name within Bluesky, via a partnership with a domain registrar [8]. Domain names have several advantages:

- We leverage the existing infrastructure of ICANN, registrars, and name servers, including for example the dispute resolution procedures for trademarks.
- Domain names are a well-known concept even among non-technical users, and they are short and simple.
- A user can move to a different server without changing their handle (see Section 3.5).
- Users do not need to host their own server to use their own domain name; they only need to maintain a DNS record.
- For organizations and people that already have a well-known domain name, using that name makes it easy for users to check that their Bluesky account is genuine. For example, the New York Times' handle is `@nytimes.com`.
- An organization can allow its staff to demonstrate their affiliation by granting them subdomains of the organization's main domain name (comparable to institutional email addresses). For example, a journalist's handle may indicate that they are at a particular news organization.
- Offering free subdomains costs very little.

Custom Feeds and Algorithmic Choice. Several decentralized social networks offer only a reverse-chronological feed of posts from accounts the user is following – a backlash against the opaque recommendation algorithms used by mainstream social networks. For example, Mastodon advertises itself as having "no algorithms or ads to waste your time" [25].

Our belief is that the problem lies not with algorithms *per se*, but rather with centrally controlled, opaque algorithms that remove user agency and prioritize user engagement over all else, e.g. by promoting controversial posts. Good recommendation algorithms can help users discover content that is relevant to them and find new accounts to follow – especially important for new users who are not yet following many accounts. They are also helpful for surfacing content on a particular topic, whereas following a user means seeing all of their posts, which might be on a mixture of topics, not all necessarily interesting to all followers. Our goal is to offer an open and diverse marketplace of algorithms in which communities can adapt the system to suit their needs, and users have more agency over how they spend their time and attention [17].

Bluesky Social PBC offers a selection of feed algorithms of its own, and also allows anybody to create their own feed generator [4]. Tens of thousands of custom feeds have already been created. A feed generator can use arbitrary criteria to select its content. For example, it may use a manually curated list of accounts, and select posts from those accounts that contain a particular hashtag or emoji character. Use of machine learning algorithms is equally possible.

3 The AT Protocol Architecture

Bluesky is the social app with the features explained in Section 2, while the AT Protocol is the underlying decentralized foundation. We maintain this separation because the AT Protocol is designed to support multiple *social modes*, not just Bluesky. For example, besides a Twitter-style microblogging app, atproto could also be used to implement Reddit-style forums, long-form blogs with comments, or domain-specific social applications such as link sharing or book reviews. The same user identity, social graph, and user data storage servers can be shared between all of these apps.

A social mode is defined by a *lexicon*, which specifies the schema of the data and the request endpoints involved [11]. Several lexicons are currently used: for example, the `com.atproto` lexicon defines the core AT Protocol concepts such as user identity (Section 3.5), the `app.bsky` lexicon defines the microblogging mode, and the `tools.ozone` lexicon supports moderation actions [12]. Anyone can define a new lexicon, allowing new social modes to coexist alongside the Bluesky social app on a shared infrastructure. The purpose of a lexicon is to provide documentation, to allow code generation and type-checking in applications, and to facilitate schema evolution for long-term compatibility and interoperability.

The biggest constraint for new social modes is that atproto is currently designed for content that users want to make public. In particular, Bluesky user profiles, posts, follows, and likes are all public. Blocking actions are also currently public; we are investigating mechanisms for making these private [9, 26]. Only a small amount of user state is currently private: muted accounts and threads, notifications and their unread status, and user preferences such as content filtering settings. Private communications (direct messages) are currently handled by a centralized service run by Bluesky Social PBC; we plan to decentralize this component in the future.

3.1 User Data Repositories

All data that a user wishes to publish is added to their *repository*, which stores a collection of *records*. Whenever a user performs some action – making a post, liking another user's post, following another user, etc. – that action becomes a record in their repository. Records are encoded in DAG-CBOR [30], a restricted form of CBOR [14], a compact binary data format. The schema of records is defined by the lexicon, and a repository may contain a mixture of records from several different lexicons, representing user actions in different social modes. Media files (e.g. images) are stored outside of the user's repository, but referenced by their CID [20] (essentially a cryptographic hash) from a record in the repository. Similarly, a reference to a record in another repository (e.g. identifying a post being liked) also includes its CID.

Each user account has one repository, and it contains all of the actions they have ever performed, minus any records they have explicitly deleted. A *Personal Data Server (PDS)* hosts the user's repository and makes it publicly available as a web service; we discuss PDSes in more detail in Section 3.2.

A user only updates their own repository; for example, if user A follows user B, this results only in a follow record in user A's repository, and no change to B's repository. To find all followers of user B requires indexing the content of all repositories. This design decision is similar to the way hyperlinks work on the web: it is easy to find all the outbound links from a web page at a given URL, but to find all the inbound links to a page requires an index of the entire web, which is maintained by web search engines.

The *AT* in atproto stands for *Authenticated Transfer*, which reflects the fact that repositories are cryptographically authenticated. The records in a repository are organized into a *Merkle Search Tree* (MST), a type of Merkle tree that remains balanced, even as records are inserted or deleted in arbitrary order [2]. After every change to a repository, the root hash of the MST is signed; the public verification key for this signature is part of the user identity described in Section 3.5. This enables an efficient cryptographic proof that a given record appears within a given user's repository. Moreover, when a user updates or deletes a record, the MST enables a proof that the old record no longer appears in the repository.

3.2 Personal Data Servers (PDS)

A PDS stores repositories and associated media files, and allows anybody to query the data it hosts via a HTTP API. Moreover, a PDS provides a real-time stream of updates for the repositories it hosts via a WebSocket. Indexers (see Section 3.3) subscribe to this stream in order to find out about new or deleted records (posts, likes, follows, etc.) with low latency. This architecture is illustrated in Figure 1.

Hosting a PDS for a small number of users requires only small computing resources, even if those users have a large number of followers. Users who wish to self-host their own PDS can therefore do so on a cheap virtual machine in the cloud, or even on a Raspberry Pi connected to their home internet router. However, we expect that most users will sign up for an account on a shared PDS run by a professional hosting provider – either Bluesky Social PBC, or another company.

Compared to choosing a Mastodon server, the user's choice of PDS hosting provider is fairly inconsequential. The PDS URL is internal to the system, and is not normally visible to users. It makes no difference whether two users are on the same PDS or different PDSes, since interaction between users goes via the indexing infrastructure in any case. A user can migrate from one PDS to another by copying their repository and media files to the new PDS, and pointing their account at the new PDS URL (Section 3.5). Even if a PDS shuts down without warning, users can upload a backup of their repository to a new PDS, and thus recover their account without losing any of their posts or their social graph.

PDS operators generally perform some basic moderation by deleting any illegal content hosted on their servers. However, PDS-level moderation is much less important than server-level moderation in Mastodon, because in atproto, the primary moderation role is taken on by separate actors in the system – the labelers and feed generators (see Section 3.4). This allows different sets of people to offer server hosting and moderation services, respectively; we believe this separation is valuable since operating a server and moderating a community require largely disjoint sets of skills [31].

3.3 Indexing Infrastructure

On the web, websites are crawled and indexed by search engines, which then provide web-wide search and discovery features that the websites alone cannot provide. The AT Protocol is inspired by

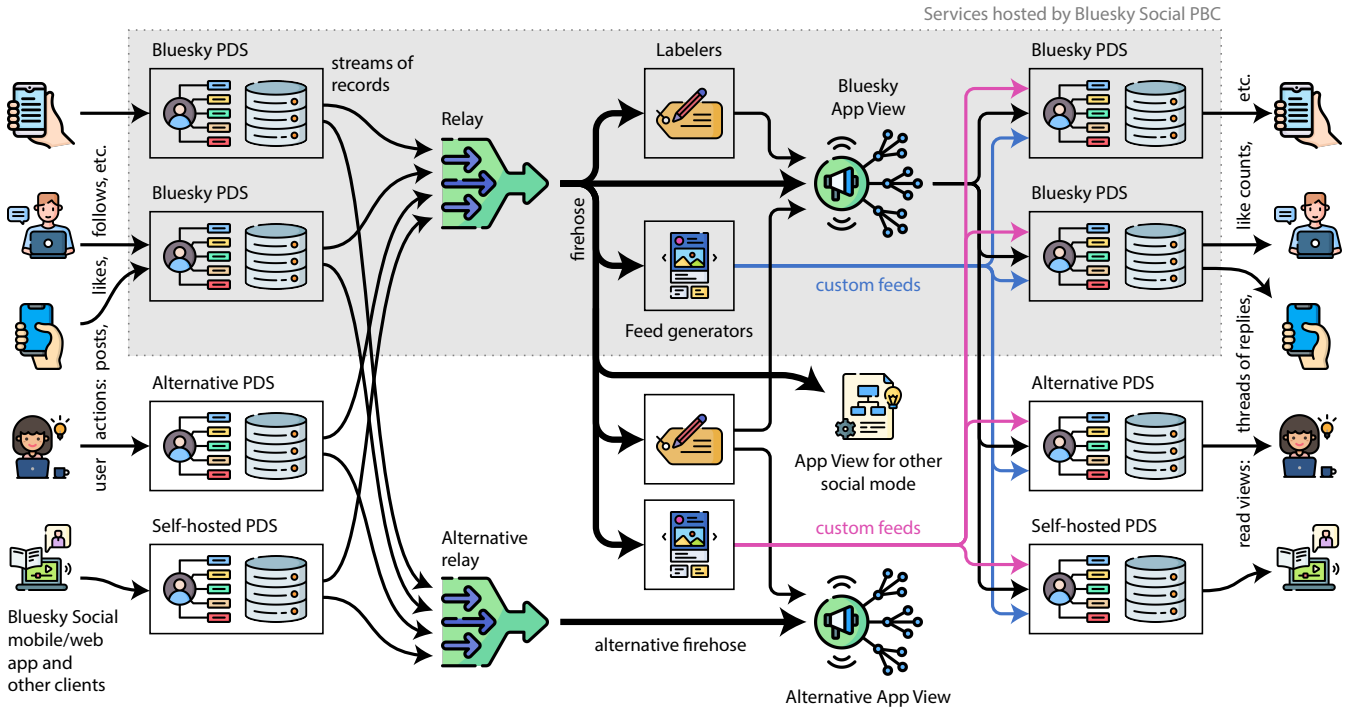

Figure 1: The main services involved in providing Bluesky, and data flows between them. Icons from Flaticon.com.

this architecture: the repositories hosted by PDSes are analogous to websites, and the indexing infrastructure is analogous to a search engine. User repositories are primary data (the "source of truth"), and the indexes are derived from the content of the repositories.

At the time of writing, most of Bluesky's indexing infrastructure is operated by Bluesky Social PBC (indicated by a shaded area in Figure 1). However, the company does not have any privileged access: since repositories are public, anybody can crawl and index them using the same protocols as our systems use. If Bluesky Social PBC were to violate users' expectations – for example, by censoring (omitting from the index) accounts that users wish to see – other parties would be free to provide their own indexes that do not perform this censorship. Client apps could switch to reading from a different index, or even use a combination of multiple indexes.

In July 2024 (when Bluesky had 6 million users), it was possible to maintain a real-time copy of all user repositories on a single server for US$153 per month [27] – expensive for a hobby project, but easily affordable by many organizations. This figure includes repository storage and the inbound bandwidth for fetching records from PDSes, but not the computational resources to build and serve summary indexes from that data.

Due to the higher cost, we expect that there will be fewer hobbyist indexers than self-hosted PDSes. Nevertheless, as Bluesky grows, there are likely to be multiple professionally-run indexers for various purposes. For example, a company that performs sentiment analysis on social media activity about brands could easily create a whole-network index that provides insights to their clients. Web search engines can incorporate Bluesky activity into their indexes, and archivists such as the Internet Archive can preserve the activity for posterity.

The indexing infrastructure operated by Bluesky Social PBC is illustrated in Figure 1. It is composed of multiple services that have integration points for external services.

3.3.1 The Relay. The first component is the *Relay*, which crawls the user repositories on all known PDSes and consumes the streams of updates that they produce. The Relay checks the signatures and Merkle tree proofs on updates, and maintains its own replica of each repository. From this information, the Relay creates the *firehose*: an aggregated stream of updates that notifies subscribers whenever records are added or deleted in any of the known repositories. Normally the WebSocket connection to PDSes provides the relay with low-latency notifications of repository changes, but in case of network interruptions the relay can also periodically re-crawl repositories and compare them to its local replica to determine what records have been added or deleted.

The firehose is publicly available. Consuming the firehose is an easier way of building an index over the whole network, compared to directly subscribing to the source PDSes, since the Relay performs some initial data cleaning (discarding malformed updates, filtering out illegal content and high-volume spam). The firehose includes Merkle proofs and signatures along with records, allowing subscribers to check that they are authentic.

The Relay does not interpret or index the records in repositories, but simply stores and forwards them. Any developers wanting to create a new social mode on top of atproto can define a new lexicon with new record types, and these records can be stored in existing

repositories and aggregated in the firehose without requiring any changes to the Relay.

3.3.2 The App View. The App View is a service that consumes the firehose, and processes the records that are relevant to the Bluesky social app (records in the `com.atproto` and `app.bsky` lexicons). For example, the App View counts the number of likes on every post, and it collates the thread of replies to each post. The App View also maintains the set of followers for each user, and constructs the timeline containing the posts by the accounts that each user is following. It then offers a web service through which this information can be queried. When a record contains references to images, the App View fetches those files from the original PDS, resizes them if necessary to reduce the file size, and makes them available via a content delivery network (CDN).

To display this information in the user's client app, the client queries the user's own PDS, which then fetches the necessary data from an App View. The App View is also responsible for enforcing moderation controls: for example, if one user has blocked another, and one of the users' repositories contains a record of an interaction that should not have been allowed due to the block, then the App View drops that interaction so that nobody can see it in the client apps. This behavior is consistent with how blocking works on Twitter/X [34], and it is also the reason why blocks are public records in Bluesky: every protocol-conforming App View needs to know who is blocking who in order to enforce the block [9, 26]. If users are unhappy with the moderation rules applied by the App View operated by Bluesky Social PBC, it is always possible for third parties to operate alternative App Views that index the same firehose and present the data in a different way.

Other social modes built upon atproto require their own App View services to interpret and index the records in users' repositories as required. The PDSes and relay can be reused.

3.4 Labelers and Feed Generators

Relay and App View aim to provide a mostly "unopinionated" service: they compute indexes over repositories in a neutral way, without attempting to rank or classify content. However, a good user experience also requires "opinionated" judgements for the purposes of content filtering (e.g. detecting sexually explicit images or spam) and curation (e.g. selecting posts on a particular topic).

The AT Protocol separates out the "opinionated" aspects of the system into separate services: *labelers* and *feed generators*. These services typically take the firehose as their input. Labelers produce a stream of judgements about content (e.g. "this post is spam"), whereas feed generators return a list of post IDs they have selected for inclusion in a custom feed. Users can choose in their client app which feeds and which labelers they want to use. The output of labelers is consumed by App Views or PDSes in order to apply content filtering [6]. For a feed generator, an App View expands the post IDs into full posts before sending them to the client app of users who have subscribed to that feed.

Having labeler and feed generator services that are separate from App Views has several advantages:

- Anyone can run such services, which enables a pluralistic ecosystem in which different parties may make different judgements about the same piece of content. Users, as well as the operators of App Views and PDSes, can decide whose judgements they want to trust, and it is easy for them to switch to alternative labeling and feed generation services if their current providers fail to meet their expectations.
- It becomes easier to set up alternative App View providers: since any App View can consume the publicly available output from labelers and feed generators, there is less pressure for each App View to develop its own content filtering infrastructure. Having alternative App Views is important for a healthy, decentralized marketplace.

Feed generators can be implemented using our starter kit [5], or using a third-party service such as Skyfeed [7].

3.5 User Identity

The DNS-based user identity in atproto is decoupled from hosting and indexing. Anybody owning a domain name can issue subdomains as atproto handles, and this does not require running a PDS or any other service besides DNS.

Handles can change, but every Bluesky/atproto account also has an immutable, unique identifier: a *decentralized ID* or *DID*, which is a URI starting with the prefix "`did:`". For example, when a record in user A's repository indicates that A is following B, that record contains B's DID; this allows a user to change their handle without affecting their social graph. DIDs are a recent W3C standard [32].

Moreover, we want a user to be able to migrate to a different PDS without changing either their DID or their handle. DIDs provide a mechanism for *resolving* a DID into a *DID document*, a JSON document containing information about the user identified by that DID, as illustrated in Figure 2. In atproto, a DID document specifies the handle of the user, the URL of their PDS, and the public key that is used to sign the Merkle tree root of their repository every time they add or delete a record. To change their handle or their PDS, the user needs to update their DID document to the new value.

To prove ownership of a handle, the user must have a bidirectional link between their DID and their domain name handle, as shown in Figure 2:

- A link from the handle to the DID is established either by storing the DID in a DNS TXT record on that domain name, or by returning the DID in response to a HTTPS request to a `/.well-known/` URL on that domain name [23].
- A link from the DID to the handle is established by including the handle in the DID document that is returned when the DID is resolved.

The App View periodically checks these bidirectional links, and invalidates the handle if either is broken. Provided that the App View is honest and takes measures to protect against DNS poisoning attacks (perhaps using DNSSEC when available), this approach prevents users from impersonating domains that they do not own.

3.5.1 Resolving DID documents. The DID specification [32] does not directly specify the mechanism for resolving a DID into a DID document. Rather, the first substring after `did:` in a DID indicates the *DID method*, and the specification of the DID method defines the protocol for obtaining the DID document. Hundreds of DID methods have been defined [33], many of which are dependent on specific blockchains or other external systems. To avoid atproto

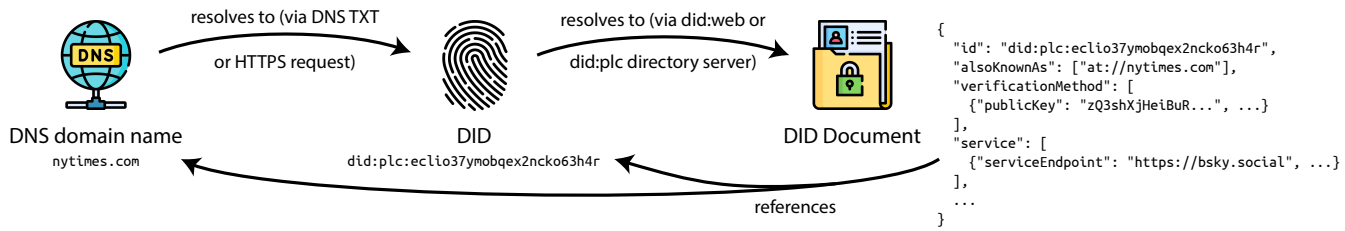

Figure 2: A handle resolves to a DID, and a DID resolves to a DID document, which in turn references the handle, DID, and the user's public key. Icons from Flaticon.com.

implementations having to support so many resolution methods, our services currently only accept DIDs based on either `did:web` (defined by the the W3C Credentials Community Group [18]) or `did:plc` (defined by ourselves for atproto [19]). Support for more DID methods might be added in the future.

The `did:web` method is very simple: the part of the DID after `did:web:` is a domain name, and the DID document is resolved by making a HTTPS request to a `/.well-known/` URL on that domain name. The security of a `did:web` identity therefore assumes that the web hosting provider for that domain is trusted, and also relies on trusting the TLS certificate authorities that may authenticate the HTTPS request.

`did:web` identities are therefore similar to domain name handles, with the difference that the name cannot be changed, since a DID is an immutable identifier. This makes `did:web` appropriate for the identity of organizations that are already strongly linked to a particular domain name. For most users, `did:plc` is more appropriate, since it uses a domain name only as a handle that can be changed.

3.5.2 The did:plc DID method. When a user creates an account on the Bluesky social app, they are by default assigned a DID of the form `did:plc:eclio37ymobqex2ncko63h4r`, where the string after the prefix `did:plc:` is the SHA256 hash of the initial DID document, truncated to 120 bits and encoded using base32 [19]. A DID of this form can be resolved to the corresponding DID document by querying a server at https://plc.directory/, which is currently operated by Bluesky Social PBC; in the future we plan to establish a consortium of independent operators that collectively provide the PLC directory service.

The PLC directory server plays an authoritative role similar to the DNS root servers, but it is mostly untrusted because PLC DID documents are self-certifying. If the DID document has not changed since its initial creation, it is easy to verify that a DID has been correctly resolved to a DID document by recomputing its hash. To support changes to the DID document, the initial version of a user's DID document contains a public key that is authorized to sign a new version of the DID document. Any new version of the DID document is only valid if it has been signed by the key in the previous version. The directory returns all DID document versions for a given DID, allowing anybody to check the chain of signatures.

If the directory server were to be malicious, it would not be able to modify any DID documents – it could only omit valid DID document versions from its responses, or fail to respond at all. Moreover, if there were to be a fork in DID document history such that two correctly signed successor versions for some DID document exist, the directory server could choose which one of these forks to serve. To mitigate such attacks, we anticipate that a future version of the PLC directory will use an append-only transparency log similar to certificate transparency [21].

3.5.3 Authentication. In principle, the cryptographic keys for signing repository updates and DID document updates can be held directly on the user's devices, e.g. using a cryptocurrency wallet, in order to minimize trust in servers. However, we believe that such manual key management is not appropriate for most users, since there is a significant risk of the keys being compromised or lost.

The Bluesky PDSes therefore hold these signing keys custodially on behalf of users, and users log in to their home PDS via username and password. This provides a familiar user experience to users, and enables standard features such as password reset by email. The AT Protocol does not make any assumptions about how PDSes authenticate their users; other PDS operators are free to use different methods, including user-managed keys.

4 Conclusions

Bluesky and the AT Protocol are a new approach to social media. Their architecture is based on the principle that every part of the system can be run by multiple competing providers, and users can switch providers with minimal friction (in particular, without changing username, and without losing any of their content or social graph). For example, anyone can write a client, host a PDS, index the network, or provide moderation services, and all of these services interoperate. Even though the majority of Bluesky services are currently operated by a single company, we nevertheless consider the system to be decentralized because it provides *credible exit* [28]: if Bluesky Social PBC goes out of business or loses users' trust, other providers can step in to provide an equivalent service using the same dataset and the same protocols.

While some decentralized systems prioritize censorship resistance, we believe that a good user experience requires explicitly addressing problematic content such as harassment and misinformation. We therefore make moderation a first-class concern that is handled separately from infrastructure hosting, and we provide strong mechanisms for users to control the content they see. Our open architecture allows a pluralistic system in which different users may choose different providers that uphold different values, while still allowing them to communicate and interoperate.

References

[1] Ben Adida. 2022. Don't let federation make the experience suck. https://benlog.com/2022/12/28/dont-let-federation-make-the-experience-suck/ Archived at https://perma.cc/W7CY-TF23.

[2] Alex Auvolat and François Taïani. 2019. Merkle Search Trees: Efficient State-Based CRDTs in Open Networks. In *38th Symposium on Reliable Distributed Systems (SRDS 2019)*. IEEE, 221–230. https://doi.org/10.1109/srds47363.2019.00032

[3] Chelsea Barabas, Neha Narula, and Ethan Zuckerman. 2017. *Defending Internet Freedom through Decentralization: Back to the Future?* Technical Report. MIT Media Lab. https://dci.mit.edu/decentralizedweb Archived at https://perma.cc/Q8CJ-D44Y.

[4] Bluesky Social PBC. 2023. Algorithmic Choice with Custom Feeds. https://blueskyweb.xyz/blog/7-27-2023-custom-feeds Archived at https://perma.cc/Z6U4-VMY8.

[5] Bluesky Social PBC. 2023. ATProto Feed Generator. https://github.com/bluesky-social/feed-generator

[6] Bluesky Social PBC. 2023. Bluesky Proposal 0002: Labeling and Moderation Controls. https://github.com/bluesky-social/proposals/tree/main/0002-labeling-and-moderation-controls

[7] Bluesky Social PBC. 2023. Featured Community Project: SkyFeed. https://atproto.com/blog/feature-skyfeed Archived at https://perma.cc/AYR8-AY5K.

[8] Bluesky Social PBC. 2023. Purchase and Manage Domains Directly Through Bluesky. https://blueskyweb.xyz/blog/7-05-2023-namecheap Archived at https://perma.cc/QUA7-L8QJ.

[9] Bluesky Social PBC. 2023. Why are blocks on Bluesky public? https://atproto.com/blog/block-implementation Archived at https://perma.cc/2ZQX-KTNJ.

[10] Bluesky Social PBC. 2024. AT Protocol Community Projects. https://atproto.com/community/projects Archived at https://perma.cc/X88A-9XM4.

[11] Bluesky Social PBC. 2024. AT Protocol Specification. https://atproto.com/specs/atp

[12] Bluesky Social PBC. 2024. atproto lexicons. https://github.com/bluesky-social/atproto/tree/main/lexicons

[13] Bluesky Social PBC. 2024. GitHub repositories. https://github.com/bluesky-social

[14] Carsten Bormann and Paul Hoffman. 2020. RFC 8949: Concise Binary Object Representation (CBOR). IETF Standards Track. https://datatracker.ietf.org/doc/html/rfc8949

[15] ENS Labs Limited. 2024. Ethereum Name Service. https://ens.domains/

[16] Gildásio Filho. 2024. deck.blue. https://deck.blue/

[17] Jay Graber. 2023. Algorithmic choice. https://blueskyweb.xyz/blog/3-30-2023-algorithmic-choice Archived at https://perma.cc/WQR6-5QJF.

[18] Christian Gribneau, Michael Prorock, Orie Steele, Oliver Terbu, Mike Xu, and Dmitri Zagidulin. 2023. did:web Method Specification. W3C Credentials Community Group. https://w3c-ccg.github.io/did-method-web/ Archived at https://perma.cc/WB8M-8ECW.

[19] Daniel Holmgren, Bryan Newbold, Devin Ivy, and Jake Gold. 2023. DID PLC Method (did:plc). https://github.com/did-method-plc/did-method-plc

[20] IPFS Documentation. 2022. Content Identifiers (CIDs). https://docs.ipfs.tech/concepts/content-addressing/ Archived at https://perma.cc/65PP-ZRQW.

[21] Ben Laurie. 2014. Certificate Transparency. *ACM Queue* 12, 8 (Aug. 2014), 10–19. https://doi.org/10.1145/2668152.2668154

[22] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. https://www.w3.org/TR/2018/REC-activitypub-20180123/

[23] Emily Liu. 2023. How to set your domain as your handle. https://blueskyweb.xyz/blog/4-28-2023-domain-handle-tutorial Archived at https://perma.cc/4LNR-6YC5.

[24] Mike Masnick. 2019. *Protocols, Not Platforms: A Technological Approach to Free Speech.* Technical Report. Knight First Amendment Institute at Columbia University. https://knightcolumbia.org/content/protocols-not-platforms-a-technological-approach-to-free-speech Archived at https://perma.cc/2V36-FKV3.

[25] Mastodon gGmbH. 2024. Mastodon. https://joinmastodon.org/

[26] Bryan Newbold. 2023. Mechanisms for private "block" relationships between Bluesky accounts. https://github.com/bluesky-social/atproto/discussions/1131 Archived at https://perma.cc/2FWX-NPAX.

[27] Bryan Newbold. 2024. Notes on Running a Full-Network atproto Relay (July 2024). https://whtwnd.com/bnewbold.net/3kwzl7tye6u2y Archived at https://perma.cc/2FHU-D88M.

[28] Bryan Newbold. 2024. Progress on atproto Values and Value Proposition. https://bnewbold.net/2024/atproto_progress/ Archived at https://perma.cc/LC9R-Q6JY.

[29] Samuel Newman. 2024. Graysky: Bluesky, like you've never seen it before. https://graysky.app/

[30] Protocol Labs. 2021. Specification: DAG-CBOR. https://ipld.io/specs/codecs/dag-cbor/spec/ Archived at https://perma.cc/D7UV-EUFL.

[31] Yoel Roth and Samantha Lai. 2023. Collective Security in a Federated World. In *Scaling Trust on the Web*. Atlantic Council, Chapter Annex 5. https://www.atlanticcouncil.org/in-depth-research-reports/report/scaling-trust/ Archived at https://perma.cc/CT3R-DCF5.

[32] Manu Sporny, Dave Longley, Markus Sabadello, Drummond Reed, Orie Steele, and Christopher Allen. 2022. Decentralized Identifiers (DIDs) v1.0: Core architecture, data model, and representations. W3C Recommendation. https://www.w3.org/TR/did-core/

[33] Orie Steele and Manu Sporny. 2023. DID Specification Registries: The interoperability registry for Decentralized Identifiers. W3C DID Working Group. https://w3c.github.io/did-spec-registries/#did-methods Archived at https://perma.cc/LM4T-JTZ5.

[34] X Help Center. [n. d.]. How to block accounts on X. https://help.twitter.com/en/using-x/blocking-and-unblocking-accounts Archived at https://perma.cc/VZZ6-CSCM.

[35] Douglas Yeung. 2023. The 'Digital Town Square' Problem. The RAND Blog. https://www.rand.org/blog/2023/01/the-digital-town-square-problem.html Archived at https://perma.cc/3GM7-3VPP.